Figure 14.19 An example graph where A* provides a significant speedup over Dijkstra. While this is an edge case, in many situations we have some domain knowledge that A* can leverage to prune search branches and get a significant speedup.
Notice that here the straight-line distance between vertices, used as heuristic, as well as the edge weights, are expressed in multiples of a generic unit denoted by x (it could be meters, or miles, etc…).

So, from these considerations, it should be clear that A* hasn't been developed to provide a generic method to be parameterized. And yet, it can be extremely useful.

As we had mentioned, in fact, there are at least two good reasons to implement A*, two contexts in which A* provides an advantage over Dijkstra's.

Let's be clear from the beginning: this is not always true, in the general case Dijkstra is asymptotically as fast as A* (or the latter might not even meaningfully applicable).

A* gains an advantage only in some contexts where we have extra information that we can somehow use.

The first case where we can use A* to drive search faster to the goal is when we have information about the distance from all or some vertices to the goal(s). Figure 14.19 explains this situation better than a thousand words! Notice that in this particular case, the key factor is that the vertices, modeling physical places in the real world, carry extra information with them (their position, which is fixed) that can help estimating their distance to the final goal; this isn't